Physics-Constrained Optimal Transport + Diffusion + RL

Real-data adaptation of synthetic LiDAR observations • Code implementation specification

Scope: one generative model (LidarDM architecture) and one real dataset (Pandar128); no third-party pretrained checkpoints. This revision adds conditional/unbalanced OT, RL controls, baselines, evaluation gates and runnable-style pseudocode.

1. Executive decision

Build a LiDAR-first PoC that fine-tunes a conditional diffusion generator on legally authorized real LiDAR scans, optimizes domain alignment using physically meaningful optimal transport losses, and optionally learns context-dependent sampling controls with reinforcement learning. The business success criterion is downstream real-world transfer, not minimal distribution distance alone.

Layer	Choice	Role
Generator	LidarDM source architecture (MIT)	Synthetic LiDAR representation; train new checkpoints
Real domain	Pandar128 LiDAR dataset (publisher MIT)	Real calibration, development, held-out evaluation
Alignment	Conditional Sinkhorn; unbalanced OT for mass/return counts	Compare sensor response at matched conditions
Controller	SAC or contextual bandit (experimental)	Select generation parameters and scenarios
Baseline	Fixed controls; domain randomization; CMA-ES/BO	Show incremental value of RL

2. Commercial and licensing gates

Publisher statements: LidarDM source repository carries an MIT LICENSE; its documented pretrained checkpoints rely on separately governed data, including Waymo/KITTI-360. Do NOT use pretrained weights as commercially cleared by default. The Pandar128 repository and dataset license file explicitly state MIT including permission to use, modify and sell dataset copies, subject to notice terms. This is a publisher license statement, not a legal warranty that all upstream rights are cleared.

- Pin exact model commit and dataset revision; archive LICENSE, dataset card, commit SHA, and checksums.
- Use newly trained weights on approved data; review Python package, submodule, and asset licenses before distribution.
- Check data provenance and privacy, especially embedded camera images; LiDAR-only training reduces image privacy exposure.

- Do not redistribute model weights or data until provenance and contractual permissions have been reviewed.

3. Precise problem formulation

Let S be a physical scene, c a sensor configuration and Z observations. We seek a conditional generator G_phi(Z_sim, S, c, ξ) that approximates P_real(Z | S,c). When identical real/sim scenes are unavailable, replace scene conditioning with measurable scenario strata k (range, road type, surface, incidence, sensor mode). These strata control confounding but do not prove true conditional matching.

Diffusion training: $L_{diff} = E_{z0, \varepsilon, t, c}[|| \varepsilon - \varepsilon_{phi}(z_t,t,c)||^2]$.

OT training: $L = L_{diff} + \lambda_{ot} L_{COT} + \lambda_{mass} L_{UOT} + \lambda_{geo} L_{geometry} + \lambda_{temp} L_{temporal} + \lambda_{reg} L_{regularization}$.

Task success: choose model using labeled real development data where available; report final results on disjoint real test sessions. No final-test leakage into RL reward, OT fitting, or hyperparameter selection.

4. Optimal transport objective and constraints

For empirical distributions a and b with cost matrix C_ij, balanced OT solves $\min_{\Gamma \geq 0} \langle \Gamma, C \rangle$ subject to $\Gamma 1 = a$ and $\Gamma^T 1 = b$. Entropic Sinkhorn adds $\varepsilon KL(\Gamma || ab^T)$. Sinkhorn divergence subtracts self-interaction terms to reduce entropy-induced bias.

Unbalanced objective: $\min_{\Gamma \geq 0} \langle \Gamma, C \rangle + \varepsilon KL(\Gamma || ab^T) + \tau_s KL(\Gamma 1 || a) + \tau_r KL(\Gamma^T 1 || b)$. The mass penalties permit missing returns and clutter; separate return-rate metrics remain necessary because normalized feature measures erase count differences.

Conditional OT: $L_{COT} = \Sigma_k w_k SD_{\varepsilon}(F_{real} | k, F_{gen} | k)$. Fix strata before training, require minimum samples per bin and report unsupported bins. Never mix unrelated scenes and interpret a low OT score as improved sensor physics.

5. LiDAR feature representation and physical cost

Feature	Representation / cost scale	Constraint
Range	Meters, normalize by fixed calibration-domain scale	No negative range; sensor max-range bounds
Intensity	Calibrated reflectance or normalized raw intensity	Avoid assuming transfer across different intensity calibrations
Return mask	Binary valid/invalid per ring and azimuth	Retain explicit return probabilities
Point density	Count per range/angle region	Preserve scanning geometry
Temporal persistence	Return correlation across aligned sweeps	Account for ego-motion and occlusion
Scene geometry	Point-to-surface or voxel occupancy on synthetic scene	No large structural deformations

Implement transport on fixed physical feature vectors, not unstructured whole scans initially. A dimensionless cost can be C_ij = Σ_m w_m ((f_im−f_jm)/s_m)^2, with s_m fixed before training. If labels are unavailable, validate scene preservation through known simulator geometry and registration residuals.

6. Data preparation and split design

- Process Pandar128 native scans into calibrated range image [R, A, channels], mask, ring index and timestamps; retain exact inverse projection and metadata.
- Partition by complete driving traces (not random frames): e.g. training ~60%, development ~20%, final test ~20%; adjust to trace count, weather and scene representation.
- Fit normalization and OT reference features only on training/calibration traces; development traces for model selection; test traces untouched.
- Build synthetic scenes with a known coordinate frame and preserve canonical simulated scene ground truth through adaptation.
- Avoid training an unconditional generator on real data and claiming scene-conditioned fidelity: conditioning must be supported and tested.

7. Proposed implementation modules

```
simreal_ot/
  config/{data,model,ot,rl,eval}.yaml
  data/{pandar_reader,range_projection,trace_split}.py
  models/{lidardm_adapter,condition_encoder,lora}.py
  losses/{conditional_ot,unbalanced_ot,scene_constraints}.py
  env/{sensor_adaptation_env}.py
  train/{train_diffusion,train_ot,train_rl}.py
  eval/{sensor_metrics,downstream,ablation}.py
  tests/{test_projection,test_ot,test_splits,test_repro}.py
```

8. Minimal OT pseudocode (PyTorch-oriented)

```
def conditional_ot_loss(real_feat, syn_feat, real_bins, syn_bins, sinkhorn):
    losses, weights = [], []
    for k in common_nonempty_bins(real_bins, syn_bins):
        xr = real_feat[real_bins == k]
        xs = syn_feat[syn_bins == k]
        if min(len(xr), len(xs)) < MIN_SAMPLES: continue
        # normalize with train-only scales; no gradient through real reference
        losses.append(sinkhorn(xs, xr.detach()))
        weights.append(min(len(xr), len(xs)))
    if not losses: raise ValueError('No supported conditional bins')
    return weighted_mean(losses, weights)

# Unbalanced OT on non-normalized voxel/return mass:
# C = pairwise_physical_cost(source_features, target_features)
# L_uot = unbalanced_sinkhorn(a_count_mass, b_count_mass, C, reg, tau_s, tau_r)
# use a library solver with explicit unit tests for non-equal total masses
```

Implementation caution: GeomLoss Sinkhorn supports differentiable weighted samples; the unbalanced objective may require a separate POT solver or custom differentiable implementation. Confirm gradient behavior and scaling before using it inside diffusion training.

9. RL formulation: useful only if decisions are sequential

Element	Definition
State s_t	Current conditional OT vector, return-rate errors, scenario descriptors, remaining budget, previous improvement
Action a_t	Bounded guidance/adaptor strength, dropout correction, chosen scenario bin, number of diffusion samples
Transition	Generate a controlled batch, update discrepancy estimates and budget
Reward	$\alpha(D_t - D_{t+1}) - \beta$ physical penalty $- \gamma$ compute cost + occasional task improvement
Episode	Fixed number of generation calls or cost budget
Algorithm	SAC for continuous actions; contextual bandit if only one choice per scenario

Avoid having the policy directly update LoRA weights each step: freeze generator/adapter during policy training, or treat adapter training as a slower outer-loop process. Train RL only if it produces material improvements over BO/CMA-ES with equal simulator-call and GPU budgets.

```
# Gymnasium-style conceptual environment
class SensorAdaptEnv(gym.Env):
    def reset(self, seed=None, options=None):
        self.theta = self.theta0.copy(); self.used = 0
        self.prev_gap = self.evaluate_train_gap(self.theta)
        return self.observe(), {}
    def step(self, action):
        self.theta = bounded_update(self.theta, action)
        syn = self.sample_controlled_batch(self.theta, seed=self.used)
        gap = self.conditional_sinkhorn(syn, self.real_train_ref)
        phys = self.physical_constraint_loss(syn)
        cost = self.compute_cost(syn)
        reward = self.prev_gap-gap - LP*phys - LC*cost
        self.prev_gap=gap; self.used+=1
        done = self.used>=self.max_calls
        return self.observe(), reward, done, False, {'gap':gap,'phys':phys}
```

10. Experimental matrix and ablations

Experiment	Modification	Decision
E0	Unadapted architecture / baseline synthetic output	Reference
E1	Train diffusion on approved real data	Benefit of real-domain learning
E2	E1 + conditional Sinkhorn	Value of OT alignment
E3	E2 + unbalanced return-count handling	Value of mass modeling
E4	E3 + geometric & temporal constraints	Scene fidelity and physical realism
E5	E4 + SAC control	RL improvement over E4, BO and CMA-ES

Hold scene mixes, random seeds, architecture capacity and compute budgets as constant as practical. Repeat generation and downstream model training across multiple seeds. Report confidence intervals by independent session or scene—not by correlated frames.

11. Metrics and failure criteria

Metric	Requirement
OT	Conditional Sinkhorn on held-out real development samples; fixed feature scaling
Mass	Return probability and point counts by range/ring, plus clutter-like outliers
Physical	Scene geometry displacement, beam/ring compliance, occlusion violations
Temporal	Ego-motion-compensated persistence, noise autocorrelation
Downstream	Real-world segmentation/detection improvement using identical label budgets
Data efficiency	Performance versus number of real training labels
Generalization	Entire held-out traces, weather and scene types
Compute	GPU hours / simulation calls per unit improvement

Kill the hypothesis if distribution mismatch falls but real-world task performance stagnates or worsens, if the generator alters simulated object geometry, or if RL fails to outperform strong non-RL optimizers at equal budgets. A low OT score is necessary for the proposed objective, not sufficient for commercial value.

12. Implementation milestones

Phase	Engineering output	Exit gate
1. Licensing/data	Frozen revisions, rights manifest, scan reader	Reproducible ingestion of a trace
2. Generator	Re-trained LidarDM-derived baseline / synthetic render	Geometry/scan checks pass
3. OT	Conditional Sinkhorn, mass-aware discrepancy, dashboards	Synthetic controlled perturbation tests recover expected direction
4. Training	Diffusion + OT + constraints	Improves development data without geometry failure
5. RL	SAC and BO/CMA-ES comparisons	Equal-budget advantage or RL removed
6. Validation	Disjoint real test, task transfer, compute costs	Pre-registered primary metric improvement

13. Sources and license records

LidarDM source and model/checkpoint documentation: https://github.com/vzyrianov/LidarDM

Pandar128 dataset repository: https://github.com/filipberanek/lane_lines_dataset

Pandar128 dataset MIT license: https://huggingface.co/datasets/filipberanek/pandar128-lane-line-detection/blob/main/LICENSE

Python Optimal Transport: https://github.com/PythonOT/POT

GeomLoss: https://github.com/jeanfeydy/geomloss

Note: Source-code and dataset publisher license statements were checked; a full commercial legal/dependency audit has not been performed. No external pretrained weights are declared cleared.